

APUNTES

Víctor Manuel Peiró Martínez

FIIS 3ºA Programación de Sistemas Distribuidos

Contenido

Tema 1 – Introducción	2
Modelos Distribuidos	2
Limitaciones de los Clústers	2
Grandes Retos Computacionales	2
Grid Computing	3
Tema 2 – Sistemas Distribuidos de Tipo Clúster	4
Introducción	4
DDPs	4
Computación Cliente-Servidor	5
Middleware	5
Llamadas y Gestión de Procesos Remotos	5
Paso de Mensajes	5
Llamada de procedimientos remotos (RPC)	6
Clústers	7
Gestión de Procesos Distribuidos	8
Migración de Procesos	8
Acceso de 2 Nodos a un Recurso	11
Virtualización	11
Tema 3 – Kubernetes y Docker	13
Kubernetes	13
Clúster de Kubernetes	13
Capa de Red: CNI	14
Uso de Kubernetes	14
Archivos YAML	14
Directorios para datos persistentes	15
Docker	16
Creación de Imágenes	16

Tema 1 – Introducción

Modelos Distribuidos

Cycle-scavaging: Modelo de computación distribuida que aprovecha los ciclos de CPU ociosos de un gran número de ordenadores para ejecutar tareas en segundo plano. Los nodos no están dedicados y pueden entrar o salir del sistema dinámicamente.

Clúster Computing: Modelo en el que varios computadores interconectados, generalmente homogéneos y dedicados, trabajan conjuntamente como si fueran un único sistema. Se utiliza para obtener alto rendimiento, alta disponibilidad o balanceo de carga.

Internet Computing: Paradigma en el que Internet se utiliza como infraestructura de computación, permitiendo que aplicaciones y servicios se ejecuten de forma distribuida a escala global. Los recursos están geográficamente dispersos y se accede a ellos a través de la red.

Grid Computing: Modelo de computación distribuida que integra recursos heterogéneos, autónomos y geográficamente dispersos, pertenecientes a distintas organizaciones, para resolver problemas de gran escala. No existe un control centralizado fuerte y se emplea principalmente en entornos científicos.

Limitaciones de los Clústers

Mantenimiento:

- La ampliación de un clúster es costosa en grandes tamaños
- Solución escalable, ¿pero hasta qué punto?

Recursos Infrautilizados:

- Los clusters son infraestructuras dedicadas
- Una organización típica desperdicia millones de ciclos de computo

Siempre existe un problema más grande

Grandes Retos Computacionales

Los problemas científicos más importantes a los que se enfrenta el ser humano que requieren del uso de recursos computacionales para resolverlos. Para enfrentarse a ellos hay que desarrollar nuevos mecanismos computacionales, hardware y software. Su resolución implica también un avance fundamental en las tecnologías de la computación.

Problemas que no se pueden resolver fácilmente con ordenadores:

- Problemas aeroespaciales
- Problemas biológicos
- E-commerce
- Ciencias de la Tierra

Grid Computing

Permite el acceso a distintos recursos computacionales geográficamente dispersos

Tipos de Recursos:

- Potencia Computacional
- Almacenamiento
- Equipos para aplicaciones específicas

Grid utiliza protocolos de Internet e ideas de computación paralela y distribuida

Soporte de computación y almacenamiento para aplicaciones que requieren gran capacidad de cómputo y requieren análisis de un gran número de datos

Tema 2 – Sistemas Distribuidos de Tipo Clúster

Introducción

Sistema multicomputador: Un sistema de memoria distribuida se compone de un conjunto de nodos de computo completo e independientes conectados por una red de comunicación

- Su rendimiento va ligado al programa de gestión o SO distribuido

DDPs

Procesamiento de datos distribuidos

Objetivo: Procesar la información de la manera mas eficiente posible aprovechando capacidades de procesamiento paralelo y replicación de unidades

- Se eligen nodos en función de sus características
- Se añaden funciones de recuperación ante errores, perdidas (datos y comms) y caídas de nodos

Características:

- **Disponibilidad:** Con múltiples sistemas, una unidad replicada puede asumir la carga de la perdida de otro
- **Compartición de Recursos:** El hardware/software mas caro se puede compartir. Bases de datos distribuidas
- **Crecimiento Incremental:** Se puede añadir sistemas nuevos para aplicaciones con mayor carga manteniendo antiguos para cosas más sencillas
- **Mayor Productividad:** Mayor rapidez de respuesta debido a que se usa divide y vencerás

Se proponen nuevos paradigmas de programación:

- **Rango de direcciones de memoria múltiple:** Comunicación de proceso por el paso de mensajes
- **Gestión de Recursos:** Cada nodo puede autogestionarse
 - Deben sincronizarse al compartir recursos. Sistemas Maestro/Esclavo
- **Interfaces de Usuario:** Cliente/Servidor

Funciones de Soporte:

- Funciones de intercambio de datos
- Capacidad de alta disponibilidad y prestaciones
- Capacidad de gestión de procesos en entornos distribuidos

Seguridad:

- Protección de datos y recursos privados
- Garantía de autenticidad
- Diversos sistemas de criptografía para intercambio de información

Computación Cliente-Servidor

Cliente: Terminales simples para que el usuario pueda manejar

Servidor: Sistema mas complejo para ofrecer servicios compartidos entre clientes

API: Conjunto de funciones y programas que permite comunicación Cliente-Servidor

Middleware: Conjunto de controladores, APIs y software que facilitan el uso

Tipos de Aplicaciones Cliente/Servidor:

- **Procesamiento tipo host:** Toda la carga se encuentra en el servidor, el cliente solo es un terminal
- **Procesador tipo servidor:** El cliente proporciona una interfaz gráfica con las opciones a realizar y mediante mensajes al servidor este realiza las operaciones
- **Procesamiento basado en cliente:** Toda la carga recae en el cliente, el servidor se usa para tareas de validación de datos
- **Procesamiento Cooperativo:** Tanto clientes como servidores realizan trabajo repartíéndose el trabajo

Middleware

Debido a la dificultad para estandarizar todos los aspectos de la computación distribuida, se desarrollan herramientas que dan una visión uniforme a través de todas las plataformas.

- Misma forma de acceso independientemente de donde se encuentren

Middleware: Interfaces de programación que se encuentran entre la aplicación, el sistema de comunicaciones y el sistema operativo

Llamadas y Gestión de Procesos Remotos

Paso de Mensajes

Para sincronizar procesos en SSDD, se usan técnicas basadas en el paso de mensajes

Se definen una serie de funciones:

- **SEND:** Mandar mensajes para ejecutar la petición y un identificador de nodo
- **RECEIVE:** Recepción del mensaje emitido, normalmente a través de un buffer de recepción

Middleware orientado a mensajes: Una aplicación usa una interfaz de programación para ponerse en contacto con otra. Middleware permite usar Send y Receive

Llamadas a procedimiento remoto: La interfaz es una función normal, que abstrae de las funciones Send y Receive al programador.

- La invocación implementara por debajo esas funciones

Broker de Objetos: Implementación de RPC sobre objetos, cuando un cliente necesita un servicio, manda una petición a un mediador que actúa como directorio de sistemas remotos

Las funciones de pasos de mensajes pueden ser:

- **Fiabiles:** Garantizan la entrega cuando es posible
 - Usan sistemas de transporte fiables, con control de errores, acuse de recibo transmisión y reordenamiento de mensajes desordenados
- **No Fiabiles:** No garantizan la entrega, pero reducen la complejidad y la sobrecarga de información
 - La aplicación es la implementa un protocolo de fiabilidad
- **Bloqueantes:** Paron la función llamante hasta que el mensaje ha sido enviado
- **No Bloqueantes:** Devuelven el control a la función tan pronto como sea posible
 - Usan una variable que indica el estado de la llamada y un buffer de datos de envió/recepción
 - Mientras que la variable no indique que la llamada no ha terminado, no se pueden reutilizar los búferes de datos
 - Muy importantes para solapar procesos de comunicación
 - Con un envió no bloqueante se obtiene un “recibo”, que se consultará cuando se requiera el resultado de la llamada anterior. Mientras que ese recibo no indique que tenemos respuesta, realizamos trabajo útil.

Llamada de procedimientos remotos (RPC)

RPC: Encapsulan la comunicación en un sistema distribuido permitiendo a aplicaciones abstraerse del entorno distribuido

- A ojos del programador son llamadas a funciones normales, no tiene porque conocerse la infraestructura

Ventajas:

- Permite especificar interfaces remotas como conjunto de operaciones con nombres y tipos de datos dados
- Una vez especificada una interfaz, se puede generar automáticamente el código de comunicación
- Se pueden escribir módulos de comunicación portables entre arquitecturas

Proceso de Ejecución

1. El programa realiza una llamada a un procedimiento normal
2. El espacio de direcciones del llamante se incluye un stub, que enlazara el tipo de llamada
 - Estos esqueletos incluyen un sistema de comunicación que identifica el sistema remoto y envía la petición de ejecución con los parámetros necesarios
3. En la maquina remota hay otro programa esqueleto asociado al procedimiento al que se ha llamado
4. Al llegar el mensaje, se examina y se ejecuta de forma local. Se dispone de una implementación de forma local por lo que se corresponde a una ejecución normal

Representación de parámetros:

- Los programas cliente y servidor no tienen porque estar escritos en el mismo lenguaje ni arquitectura
- Si no hay protocolo no se puede asegurar que lo enviado y lo recibido a ambos lados sea interpretado igual
- Algunas herramientas de RPC proporcionan métodos de estandarización para los tipos de datos comunes y funciones para crear registros y estructuras de datos complejas
- Los parámetros nativos de cada maquina se pueden convertir a una representación estándar

Clústers

Cluster: Conjunto de elementos interconectados que trabajan conjuntamente como una sola unidad para mejorar rendimiento, disponibilidad o análisis de datos.

Tipos de Clústers:

- Dedicados/No Dedicados
- Homogéneos/ No Homogéneos
- Prioritarios/Beowulf

Aplicaciones más comunes: Internet, Bases de Datos, Calculo Científico

Aplicaciones Prácticas:

- Renderizado de Películas
- Simulación de Fluidos
- Simulación de Dinámica Molecular
- Ciberseguridad

Estos procesos requieren de muchos procesadores o muchos datos que para un ordenador el tiempo para realizar estas tareas es demasiado grande

Configuraciones de Clústers:

- La más sencilla es tener una serie de ordenadores conectados en red de alta velocidad
- Lo más común es tener al menos un sistema de ficheros compartidos de alguna forma
 - Un nodo con el sistema de ficheros para todos
 - Un sistema de ficheros distribuido con RAID

Aspectos de diseño de SO para cluster:

- **Gestión de fallos:**
 - **Alta Disponibilidad:** Ofrece la posibilidad de que todos los nodos estén en servicio. Si se pierde un nodo se pierden las peticiones a el
 - **Tolerancia a Fallos:** Asegura que todos los procesos estén disponibles.
- **Equilibrado de cargas:** Se necesita equilibrar la carga de trabajo entre todos los nodos
 - Al añadir nuevos nodos, se deben incluir en el planificador de procesos
 - Los mecanismos de middleware que implemente el SO deben de ser capaces de reconocer nuevos servicios
- **Computación Paralela:**
 - **Compilación Paralela:** Un compilador paralelo puede determinar que partes pueden ejecutarse en paralelo para asignarse a otras partes del cluster
 - **Aplicaciones Paralelas:** El programador escribe la aplicación específica para el cluster. Usa API de paso de mensajes para mover datos entre maquinas

Gestión de Procesos Distribuidos

Migración de Procesos

Migración de Procesos: Transferencia de suficiente cantidad del estado de un proceso de un computado a otro

- **Compartición de Carga:** Migrar procesos de nodos cargados a los poco cargados
- **Rendimiento de las Comunicaciones:** Si hay procesos que interactúan mucho, seria conveniente colocarlos en el mismo nodo
- **Disponibilidad:** Mover procesos de alta durabilidad de nodos en tramite de apagado a nodos que permanezcan encendidos
- **Uso de Facilidades Especiales:** Mover procesos a nodos donde haya recursos específicos que sean necesarios

¿Quién migra el proceso?

- Si el objetivo es el equilibrado, será el módulo del sistema operativo. Este necesita estar en comunicación con otros módulos para monitorizar la carga de trabajo
- Si el objetivo es alcanzar recursos en nodos particulares, será la misma aplicación. Para ello, es la aplicación la que debería conocer el estado del sistema y conocer los recursos disponibles

¿Qué parte del proceso se migra?

- Primero se destruye el proceso en el origen y se recrea en el sistema destino.
 - Los procesos pueden tener enlaces abiertos con otros procesos que habrá que redireccionar
- La dificultad se encuentra en los datos dentro de su espacio de direcciones y los ficheros abiertos.
- Si suponemos que se encuentra en un sistema con memoria virtual, se consideran las siguientes alternativas
 - **Ambicioso (Completo):** Se transfiere todo el bloque en memoria
 - Puede ser muy completo en el tiempo
 - Mejor enfoque ya que no queda rastro en la maquina origen del proceso
 - Usado en implementaciones con puntos de recuperación
 - **Ambicioso (No Completo):** Solo se envían las paginas de memoria que se encuentren en memoria principal que hayan sido referenciadas o modificadas
 - El resto se transmite bajo demanda
 - Minimiza datos que se transfieren
 - Deja restos en la maquina origen hasta que el proceso se muera o se copie entero
 - **Precopia:** El proceso sigue ejecutándose mientras la memoria se copia al nodo destino.
 - Si hay modificaciones en memoria mientras se envían los datos, habrá que volver a copiar las modificaciones
 - No queda rastro en el nodo origen
 - Reduce el tiempo que pasa el proceso parado
 - **Copiar al referenciar:** Todas las paginas de memoria se migran bajo demanda.
 - Tiene un menor coste inicial
 - **Volcado:** Cuando se dispone de un sistema de ficheros compartido por todos los nodos, se puede volcar al disco duro.

¿Qué sucede con los mensajes y señales pendientes?

- Redireccionado desde el nodo antiguo o avisar de la nueva dirección

Threads:

- Se puede migrar hilos, manteniendo sistemas de llamadas a procedimientos remotos
- No se puede migrar toda la memoria si no se han migrado todos los threads del proceso
 - Quedaran paginas distribuidas en memoria
 - La mejor opción sería enviar las páginas bajo demanda

Ficheros Abiertos:

- Cuando se migra un proceso con fichero abiertos, se pueden dar distintos casos:
 - El proceso se migra de forma temporal a un nodo y no hay lecturas en ese tiempo
 - El proceso se migra y hace lecturas/escrituras
 - Se puede duplicar en nodo destino. Problemas si otro proceso abre el mismo fichero en la maquina origen
 - Se pueden usar caches de disco y algoritmos de coherencia de cache
 - Se pueden compartir solo los bloques de disco para evitar cargar la red de datos innecesarios

Negociación de la Migración: ¿Quién se queda con el proceso?

- Para el sistema Charlotte:
 - Se deja la responsabilidad de migración u planificación a la utilidad Starter
 - Cada proceso starter se encarga de vigilar el estado de un grupo de maquinas
 - La decisión se toma al poner de acuerdo dos procesos Starter
- **Pasos:** Starter Origen (S), Starter Destino (D), Proceso (P)
 1. Se envía un mensaje al **D** para solicitar una transferencia
 2. Si **D** está listo para recibir procesos, envía un mensaje positivo
 3. **S** comunica al kernel su decisión de migrar el **P**. El kernel le envía un mensaje de oferta a **D** con las estadísticas de **P** para que decida si puede hacer cargo
 4. Si **D** esta falto de recursos, puede rechazar la oferta. En caso de rechazo, le pasa el mensaje a **S** para que decida si acepta o no el proceso
 5. Si **S** acepta, se manda un mensaje de “Inmigrar” el proceso. El kernel de **D** reenvía la petición a **S** y reserva los recursos necesarios para recibir a **P**.

Acceso de 2 Nodos a un Recurso

Al tener varios nodos accediendo a un recurso, necesitamos un sistema para asegurar que solo uno acceda.

- No debe ocurrir que un proceso que solicita un acceso a una sección crítica sea retardado de forma infinita
- Cuando la sección crítica esta libre, cualquier proceso debería acceder sin retraso
- No se puede la velocidad relativa de los procesos o el numero de procesadores
- Un proceso debería recibir en la región crítica por un tiempo finito

Exclusión Mutua:

- Es necesario un sistema de ordenación
- Suponiendo que todos los ordenadores tienen el mismo reloj, se añade a cada mensaje una etiqueta de tiempo que permita la reordenación

Cola Distribuida:

- Cada nodo contiene un proceso que realizara acceso a los recursos en exclusión mutua. Este proceso actúa como arbitro
- Los mensajes se reciben en el mismo orden que se enviaron
- La red está completamente conectada
- Todos los nodos mantienen un vector de estado del recurso compartido, que funciona como la cola FIFO.
- **Proceso:**
 1. El nodo P_i encola en su vector local y envía un mensaje de petición a todos los nodos indicando la posición dentro del vector
 2. El resto de los nodos reciben el mensaje para encolar al nodo P_i . Si en esa posición no hay otro proceso, se acepta la posición
 3. Cuando el proceso P_i se encuentra a la salida de la cola, puede acceder a la sección de exclusión mutua
 4. Cuando acaba con la sección, se envía un mensaje al resto de nodos para que saquen a P_i del vector FIFO.
 5. El resto de los procesos recibirán el mensaje y comprobaran si es su turno para acceder.

Virtualización

Permite ejecutar varias instancias de un SO o aplicaciones simultáneamente

Motivos para virtualizar:

- Baja utilización de la infraestructura
- Incremento de los costes de la infraestructura física y en el personal de TI.
- Protección frente a contingencias insuficiente
- Mantenimiento elevando de los escritorios de usuario

Ventajas:

- Menor inversión en infraestructura
- Menor coste de mantenimiento
- Mayor flexibilidad
- Mayor seguridad y tolerancia a fallo

Hypervisor/VMM: Software que se encarga de proporcionar los mecanismos específicos de virtualización

- **Tipo 1:** Directamente sobre el hardware
- **Tipo 2:** Se ejecuta dentro del SO

Tipos de Virtualización:

- **Completa:**
 - Abstracción completa del hardware
 - Proporciona soporte para SOs
 - Proporciona emulación de CPU para operaciones privilegiadas
 - Menor rendimiento que otras alternativas
- **Asistida por Hardware:**
 - Parecida a la anterior
 - La captura de instrucciones se lleva a cabo con los mecanismos HW
 - Menor sobrecarga
- **Paravirtualización:**
 - El SO es consciente de estar ejecutando en un entorno virtualizado
 - Se comunica con un hipervisor tipo 1
 - El hipervisor dialoga con el hardware
 - Instancia de SO privilegiada comunicando con el hipervisor
 - **Ventaja:** Rendimiento
- **A nivel de SO:**
 - Permite la ejecución de múltiples instancias de un mismo SO
 - De uso en entornos compartidos
 - Poca flexibilidad

Tema 3 – Kubernetes y Docker

Kubernetes

Kubernetes: Herramienta de código abierto diseñada para facilitar el despliegue de contenidos y servicios. Basado en el uso de contenedores que aíslan las aplicaciones reaprovechando el sistema operativo en el que estén desplegadas

Provee:

- Descubrimiento de servicios y balanceo de carga
- Manipulación y Configuración del almacenamiento
- Vuelta atrás a estados anteriores
- Empaquetado automático de contenidos
- Autoreparado
- Accesos seguros

No provee:

- No despliega código fuente
- No es un middleware
- No recolecta logs

Clúster de Kubernetes

Un clúster de Kubernetes está formado por nodos de trabajo, que ejecutan aplicaciones en contenedores. Cada nodo aloja los pods

Control Plane: Administra los nodos y los pods del clúster

- En entornos de producción, el control plane se ejecuta en varios equipos y un cluster generalmente ejecuta varios nodos
- Genera tolerancia a fallos y alta disponibilidad

Container: Unidad básica de despliegue, consiste en un conjunto de aplicaciones empaquetadas listas para ejecutarse.

Container runtime: Se usa docker para la creación y configuración

Pods: Unidades de despliegue de aplicaciones manejadas por Kubernetes.

- Consisten en uno o mas contenedores que permiten compartir recursos en red
- Puede representar un conjunto de aplicaciones que trabajan en común
- **Pods que ejecutan un único contenedor:** Un nodo de red con una sola imagen ejecutando sus aplicaciones
- **Pods con varios contenedores:** Se pueden unir varios contenedores para tener una aplicación más grande
- Son volátiles. Se crean bajo demanda y se destruyen cuando no se usan

Deployments: Representan los despliegues de aplicaciones. Estan formados por uno o varios pods, ejecutándose en una o varias maquinas

- Ofrece un punto de entrada con la aplicación que puede redirigirse a alguno de los pods del sistema.

Capa de Red: CNI

Para poder conectar las instancias de docker entre los distintos pods, es necesario un programa externo

Se provee una capa CNI que cualquiera pueda implementar:

- **Weave:** Flexibilidad en la configuración de red
- **Flannel:** Facilidad de uso, muy popular
- **Calico:** Centrado en eficiencia de comunicaciones

Uso de Kubernetes

kubeadm: Sirve para gestionar la red virtual y los servicios principales de Kubernetes

- **kubeadm init:** Inicia el servicio kubernetes en el nodo máster.
 - Configura el sistema de claves para poder unirse a la red virtual
 - Inicia es servicio kubelet del nodo master para poder levantar pods/deployments en este nodo
- **kubeadm reset:** Apaga el servicio Kubernetes y destruye la red virtual
- **kubeadm join:** Se ejecuta desde los nodos esclavos. Añade un nodo a la red de Kubernetes

kubectl: Sirve para configurar aplicaciones lanzadas dentro de Kubernetes

Formato: kubectl <comando> <elemento> <ID> <opciones>

- **Comando:** La operación que queremos realizar
- **Elemento:** Objeto sobre el que se quiere realizar la operación

Crear un deployment: kubectl create deployment <nombre> --image <nombre_imagen> --replicas <num_copias>

Acceder a un pod en ejecución: kubectl exec <podID> -- <comando>

Archivos YAML

Aplicar archivo yaml: kubectl apply -f <nombre_archivo>

Campos:

- **apiVersion:** Version a usar
- **kind:** Tipo de objeto a crear
- **metadata:** Datos generales sobre el objeto
- **spec:** Estado del objeto que se quiere crear. Puede contener subcategorías

Ejemplo creación deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubernetes-bootcamp
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: bootcamp
  template:
    metadata:
      labels:
        app: bootcamp
    spec:
      containers:
        - name: kubernetes-bootcamp
          image: docker.io/jocatalin/kubernetes-bootcamp:v2
```

Ejemplo creación servicio:

```
apiVersion: v1
kind: Service
metadata:
  name: bootcamp-entripoint
  namespace: default
spec:
  type: NodePort
  selector:
    app: bootcamp
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 31000
  externalTrafficPolicy: Cluster
```

Directorios para datos persistentes

Los pods creados usan sistemas de fichero volátiles. Si se borra el pod, se pierde el trabajo

Se usan volúmenes para guardar datos que se necesiten en ejecuciones futuras

Volumen: Objeto de Kubernetes similar a un disco duro. Los volúmenes pueden estar implementados cualquier sistema de disco duro.

- **Directorios locales:** hostPath
- **Directorios en red:** nfs
- **Directorios en la nube:** AWS S3

Docker

Permite crear contenedores de aplicaciones con todas las librerías desde el sistema operativo.

Facilita la migración de aplicaciones levantando sus servicios de manera independiente dentro del SO

Kubernetes usa docker como sistema contenedor de los pods que se ejecuten en su red

Creación de Imágenes

Para crear una nueva imagen, se deben crear un directorio con los siguientes recursos

- **Dockerfile:** Archivo con las reglas de construcción de la imagen
- **Archivos extra que añadir:** Si queremos copiar archivos a la imagen docker, debemos colocarlos junto al archivo Dockerfile

Los archivos Dockerfile consisten en una serie de reglas de construcción para nuestro contenedor

- **Sección “selección de sistema base”:** Permite partir de una distribución del sistema operativo
- **Sección “instalación de programas genéricos”:** Permite añadir los programas que se usaran dentro del contenedor
- **Sección “configuración de ejecución”:** Añade configuraciones y programas externos que no se encuentren en la distribución de la que se parte